Sprawozdanie z działania programu

File Crypto - Aplikacja do szyfrowania plików

Bartłomiej Karkoszka – 094062
Filip Stępień – 094117
Rafał Grot – 094046

21 stycznia 2026

Spis treści

1 Wstęp

1.1 Cel i zakres sprawozdania

Niniejsze sprawozdanie przedstawia kompleksową analizę działania aplikacji *File Crypto* - programu do szyfrowania i deszyfrowania plików napisanej w języku Java z wykorzystaniem frameworka JavaFX. Sprawozdanie obejmuje opis architektury aplikacji, implementowanych algorytmów kryptograficznych, interfejsu użytkownika oraz wyników testów funkcjonalnych przeprowadzonych na rzeczywistych danych.

1.2 O aplikacji File Crypto

File Crypto jest nowoczesną aplikacją desktopową umożliwiającą bezpieczne szyfrowanie i deszyfrowanie plików przy wykorzystaniu trzech różnych algorytmów kryptograficznych. Aplikacja została napisana w technologii JavaFX, co zapewnia nowoczesny, intuicyjny interfejs użytkownika. Główne cechy aplikacji to:

- Wsparcie dla trzech algorytmów szyfrowania: Caesar Cipher, XOR Cipher oraz AES
- Możliwość szyfrowania wielu plików jednocześnie do pojedynczego archiwum ZIP
- Konfigurowalne parametry szyfrowania dla każdego algorytmu
- Przyjazny interfejs graficzny z wskaźnikiem postępu operacji
- Obsługa błędów i walidacja danych wejściowych
- Wyświetlanie szczegółowych statystyk po zakończeniu operacji

2 Architektura aplikacji

2.1 Struktura projektu

Aplikacja została zorganizowana zgodnie z wzorcem MVC (Model-View-Controller) oraz zasadami modularności. Struktura katalogów przedstawia się następująco:

```
file-crypto/
          src/main/java/xyz/cursedman/filecrypto/
                App.java                    # G   wna klasa aplikacji
                controllers/               # Kontrolery interfejsu
                      EncryptTabController.java
                      DecryptTabController.java
                      AlgorithmSettingsController.java
                      ...
                cryptors/                  # Implementacje algorytm w
                      CryptorAlgorithm.java
                      AESCryptor/
                      CaesarCryptor/
                      XorCryptor/
                      HeaderCryptor.java
                keys/                      # Zarz dzanie kluczami
                controls/                  # Komponenty UI
                utils/                     # Narz dzia pomocnicze
          src/main/resources/             # Zasoby aplikacji
                views/                     # Pliki FXML
          pom.xml                         # Konfiguracja Maven
```

Listing 1: Struktura projektu

2.2 Technologie wykorzystane

Tabela 1: Technologie wykorzystane w projekcie

Technologia	Wersja
Java	21
JavaFX	21
Maven	3.9+
JUnit 5	5.10.2
ControlsFX	11.2.1
BootstrapFX	0.4.0
Lombok	1.18.42

3 Algorytmy kryptograficzne

3.1 Przegląd dostępnych algorytmów

Aplikacja implementuje trzy różne algorytmy kryptograficzne, każdy o różnych właściwościach bezpieczeństwa i wydajności:

3.1.1 Caesar Cipher

Caesar Cipher jest najprostszym z implementowanych algorytmów. Polega na przesunięciu każdej wartości bajtu o stałą liczbę pozycji w zakresie 0-255. Algorytm ten charakteryzuje się prostotą implementacji, ale oferuje niski poziom bezpieczeństwa.

```
public void encrypt(InputStream in, OutputStream out) throws IOException
    {
    CaesarCryptorKey key = (CaesarCryptorKey) getKey();
    int shift = key.getShift();

    int data;
    while ((data = in.read()) != -1) {
        out.write((data + shift) & 0xFF);
    }
}
```

Listing 2: Implementacja Caesar Cipher

3.1.2 XOR Cipher

Algorytm XOR wykorzystuje operację XOR z kluczem, który może być dłuższy niż 1 bajt. Operacja XOR jest symetryczna - ta sama funkcja służy do szyfrowania i deszyfrowania. Klucz może być podany jako tekst lub w formacie heksadecymalnym.

```
private void xor(InputStream in, OutputStream out) throws IOException {
    XorCryptorKey xorKey = (XorCryptorKey) getKey();
    byte[] keyBytes = xorKey.getKey();
    int keyLen = keyBytes.length;

    int data;
    int i = 0;
```

```
    while ((data = in.read()) != -1) {
        out.write(data ^ keyBytes[i % keyLen]);
        i++;
    }
}
```

Listing 3: Implementacja XOR Cipher

3.1.3 AES (Advanced Encryption Standard)

AES jest najbardziej bezpiecznym algorytmem implementowanym w aplikacji. Wykorzystuje tryb CBC (Cipher Block Chaining) z losowo generowanym wektorem inicjalizującym (IV). Algorytm ten jest standardem przemysłowym i oferuje wysoki poziom bezpieczeństwa przy dobrej wydajności.

```
public void encrypt(InputStream in, OutputStream out) throws IOException
    {
    // Generowanie losowego IV
    byte[] iv = new byte[IV_SIZE];
    SecureRandom random = new SecureRandom();
    random.nextBytes(iv);

    Cipher cipher = Cipher.getInstance(TRANSFORMATION);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, new IvParameterSpec(iv))
   ;

    out.write(iv); // Zapis IV na pocz tku pliku

    try (CipherOutputStream cos = new CipherOutputStream(out, cipher)) {
        byte[] buffer = new byte[4096];
        int read;
        while ((read = in.read(buffer)) != -1) {
            cos.write(buffer, 0, read);
        }
    }
}
```

Listing 4: Implementacja AES

3.2 Porównanie algorytmów

Tabela 2: Porównanie algorytmów kryptograficznych

Algorytm	Bezpieczeństwo	Wydajność	Złożoność konfiguracji
Caesar Cipher	Niskie	Niska	Niska
XOR Cipher	Średnie	Średnia	Średnia
AES	Wysokie	Wysoka	Wysoka

4 Interfejs użytkownika

4.1 Główny widok aplikacji

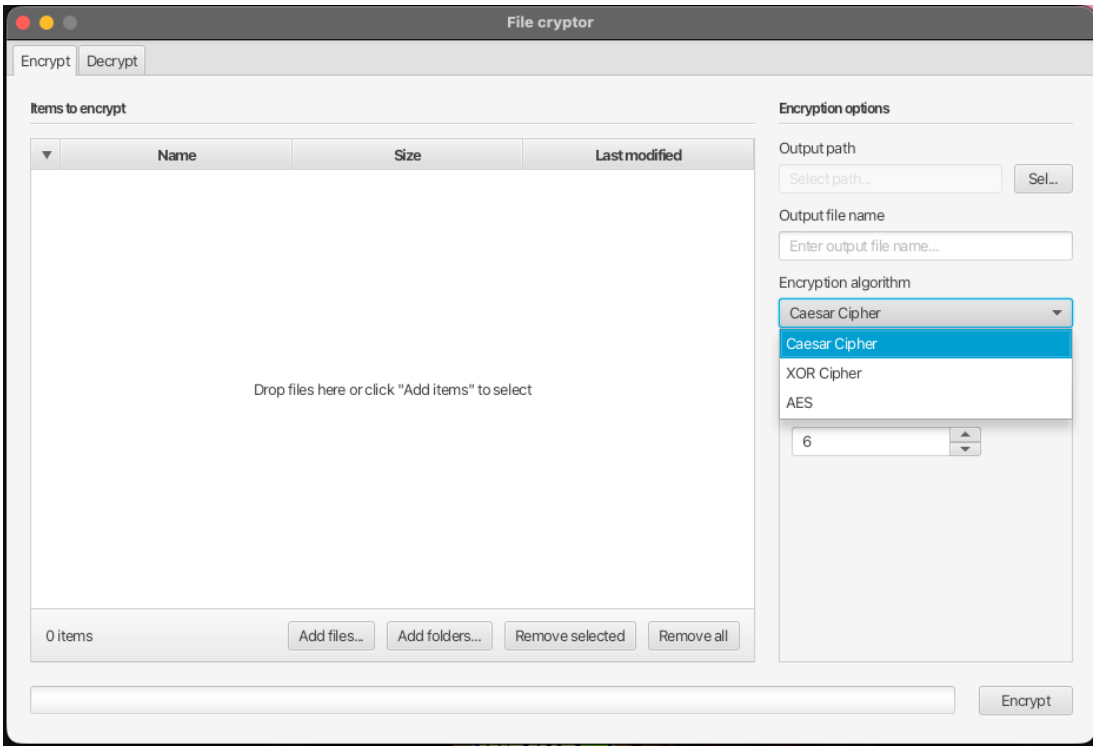
Aplikacja wykorzystuje interfejs typu TabPane z dwoma głównymi zakładkami: *Encrypt* oraz *Decrypt*. Interfejs został zaprojektowany w sposób intuicyjny, umożliwiając łatwe wykonywanie operacji szyfrowania i deszyfrowania.

4.2 Zakładka szyfrowania

Zakładka Encrypt umożliwia wybór plików do zaszyfrowania oraz konfigurację parametrów szyfrowania. Interfejs składa się z dwóch głównych sekcji: listy plików do zaszyfrowania oraz panelu konfiguracyjnego.

4.2.1 Wybór plików

Użytkownik może dodawać pliki do listy poprzez przeciąganie i upuszczanie (drag-and-drop) lub przyciski wyboru. Aplikacja wyświetla szczegółowe informacje o każdym pliku: nazwę, rozmiar oraz datę ostatniej modyfikacji.



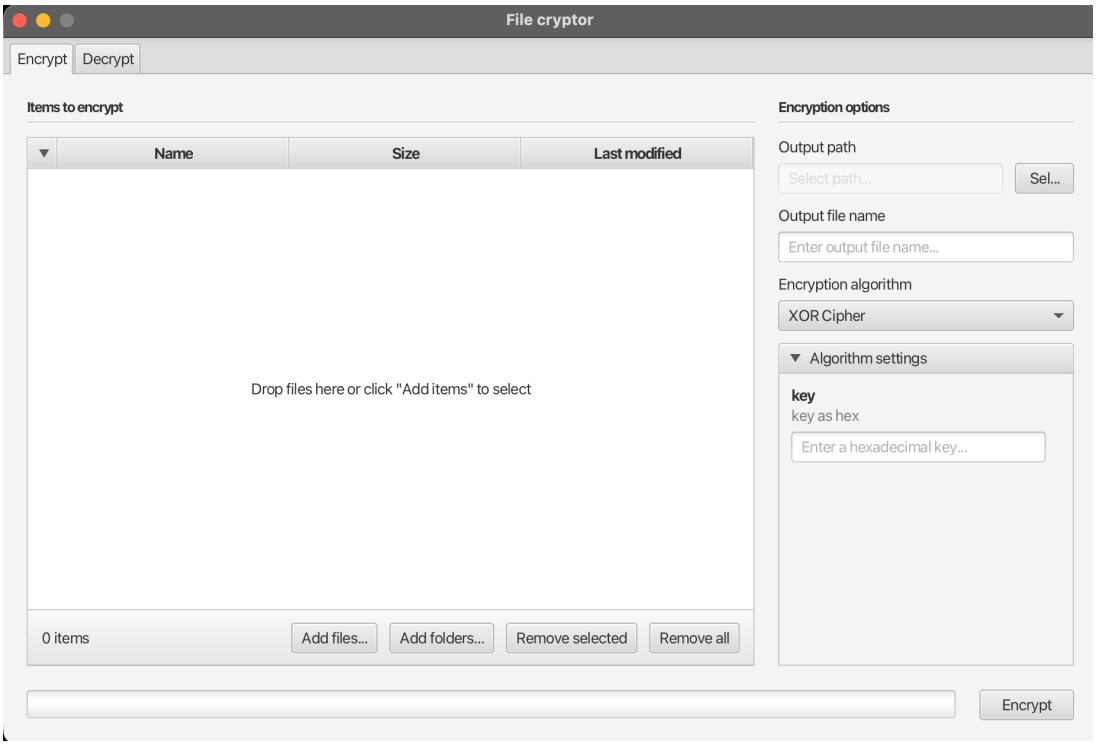
Rysunek 1: Zakładka szyfrowania z wyborem algorytmu Caesar Cipher

4.2.2 Konfiguracja algorytmu

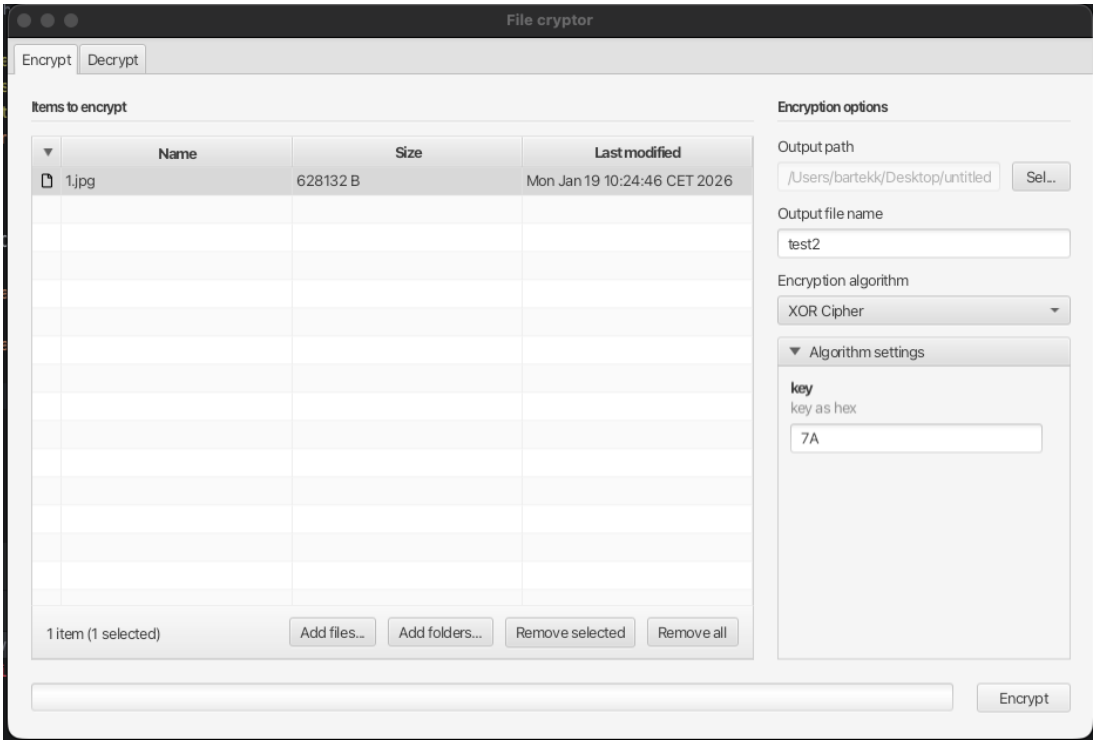
Dla każdego algorytmu dostępne są specyficzne parametry konfiguracyjne:

- **Caesar Cipher:** Parametr przesunięcia (shift) - liczba całkowita w zakresie 0-255
- **XOR Cipher:** Klucz w formacie tekstowym lub heksadecymalnym

- **AES:** Klucz w formacie heksadecymalnym (32 bajty dla AES-256)



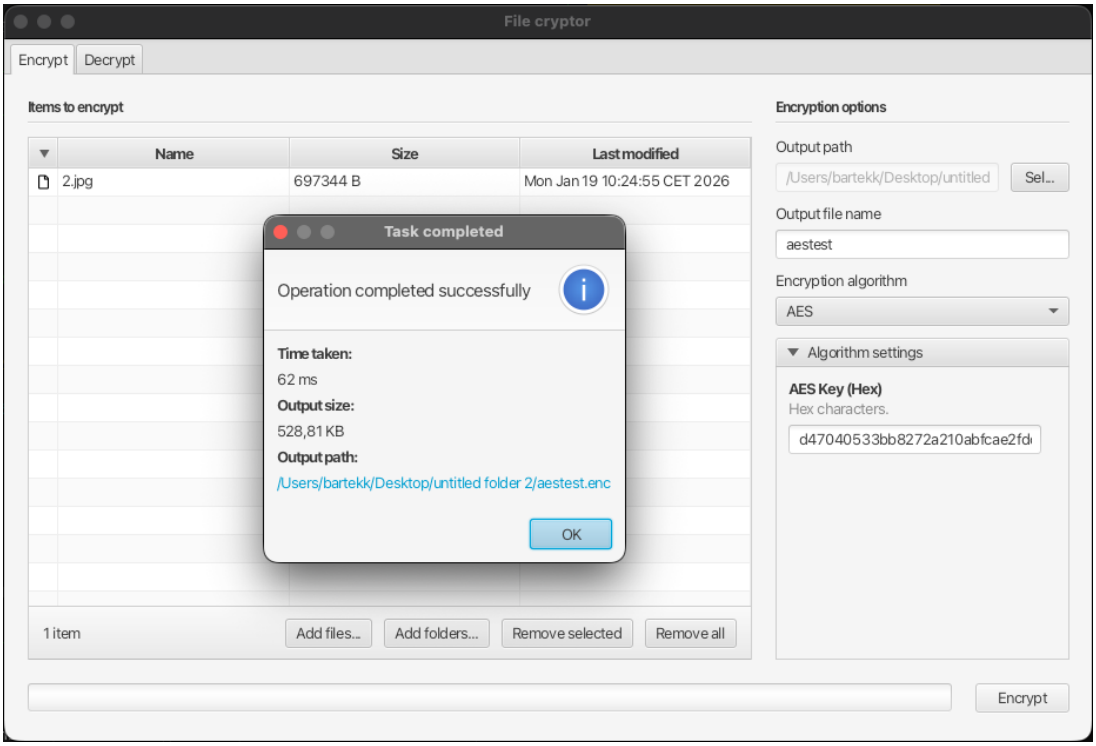
Rysunek 2: Zakładka szyfrowania z algorytmem XOR Cipher



Rysunek 3: Zakładka szyfrowania z wybranymi plikami i algorytmem XOR

4.2.3 Proces szyfrowania

Po wybraniu plików i skonfigurowaniu algorytmu, użytkownik może rozpocząć proces szyfrowania. Aplikacja wyświetla wskaźnik postępu oraz szczegółowe informacje po zakończeniu operacji.

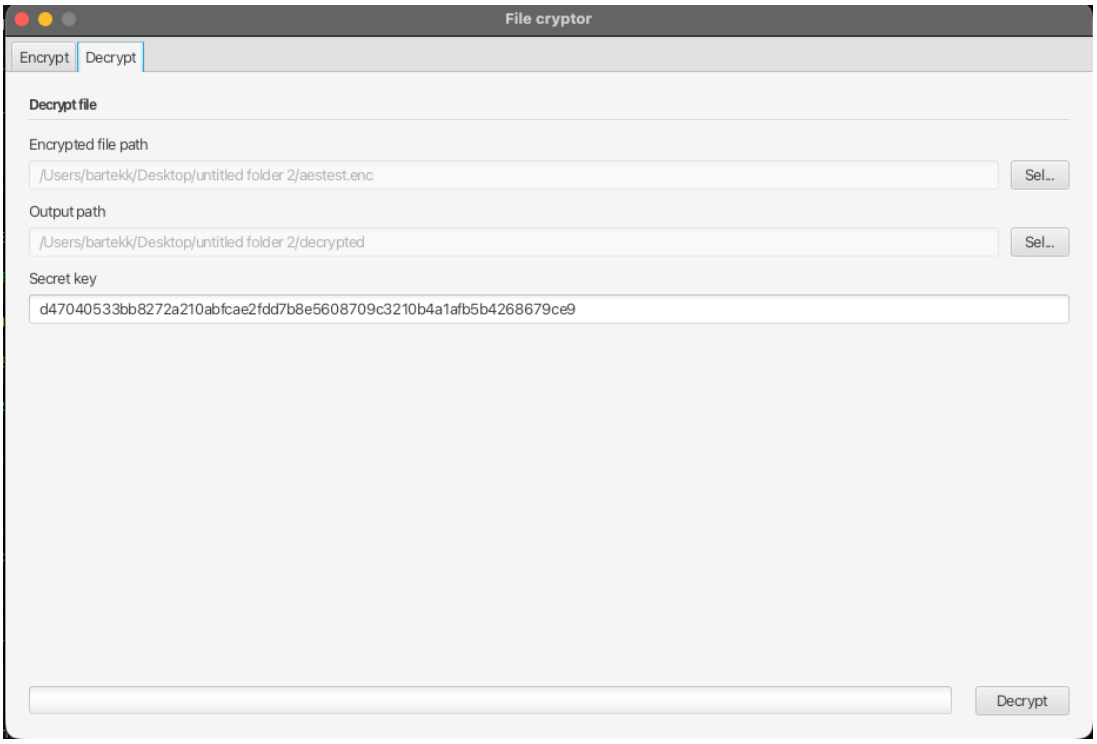


Rysunek 4: Zakładka szyfrowania z algorytmem AES

4.3 Zakładka deszyfrowania

Zakładka Decrypt umożliwia wybór zaszyfrowanego pliku i jego deszyfrowanie. Użytkownik musi podać:

- Ścieżkę do zaszyfrowanego pliku (rozszerzenie .enc)
- Ścieżkę wyjściową dla zdeszyfrowanych plików
- Klucz szyfrowania (w formacie odpowiednim dla użytego algorytmu)



Rysunek 5: Zakładka deszyfrowania z konfiguracją

5 Funkcjonalność aplikacji

5.1 Proces szyfrowania

Proces szyfrowania składa się z następujących kroków:

1. Wybór plików do zaszyfrowania (możliwość wyboru wielu plików)
2. Wybór algorytmu kryptograficznego z listy rozwijanej
3. Konfiguracja parametrów algorytmu (klucz, przesunięcie, etc.)
4. Wybór ścieżki wyjściowej i nazwy pliku
5. Uruchomienie procesu szyfrowania (wykonywane w osobnym wątku)
6. Wyświetlenie wyników operacji z czasem wykonania i rozmiarem pliku

Aplikacja automatycznie tworzy archiwum ZIP zawierające wszystkie wybrane pliki, a następnie szyfruje całe archiwum przy użyciu wybranego algorytmu.

5.2 Obsługa błędów

Aplikacja zawiera kompleksowy system obsługi błędów:

- Walidacja danych wejściowych (sprawdzanie czy pliki istnieją, czy ścieżki są poprawne)
- Kontrola poprawności kluczy szyfrowania (format, długość)

- Obsługa błędów wejścia/wyjścia
- Wyświetlanie komunikatów błędów użytkownikowi w czytelnej formie

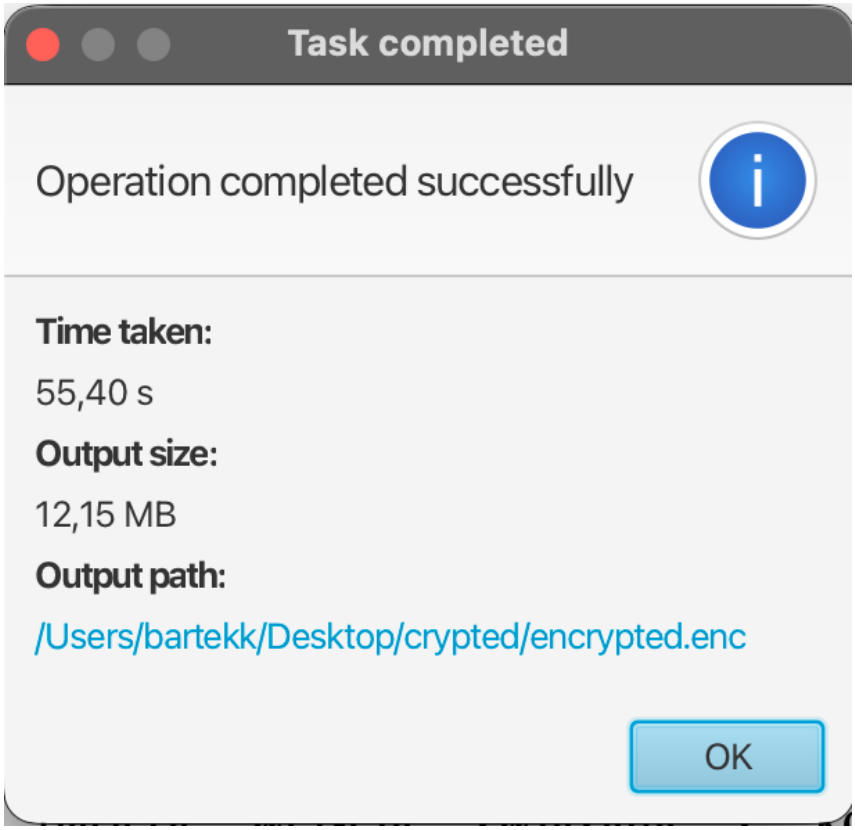


Rysunek 6: Obsługa błędu deszyfrowania - nieprawidłowy klucz

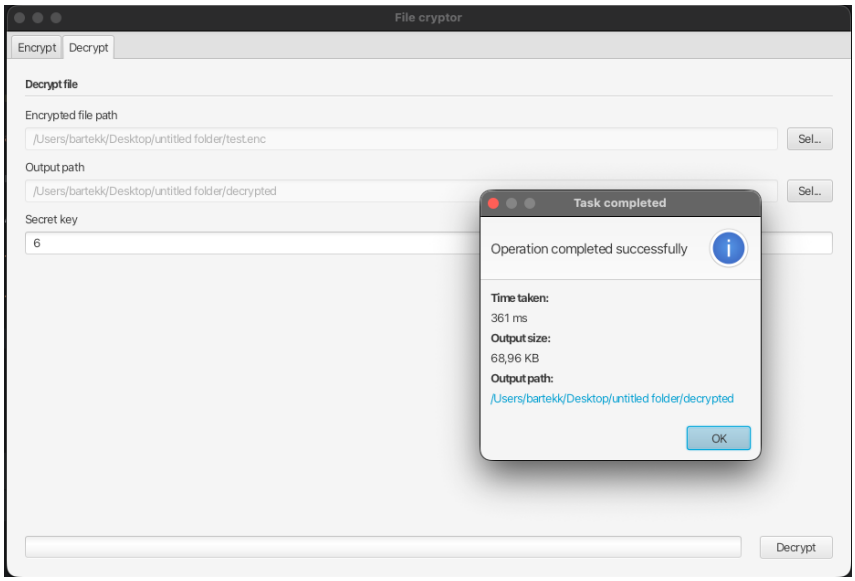
5.3 Komunikaty sukcesu

Po pomyślnym zakończeniu operacji, aplikacja wyświetla szczegółowe informacje o wykonanej operacji:

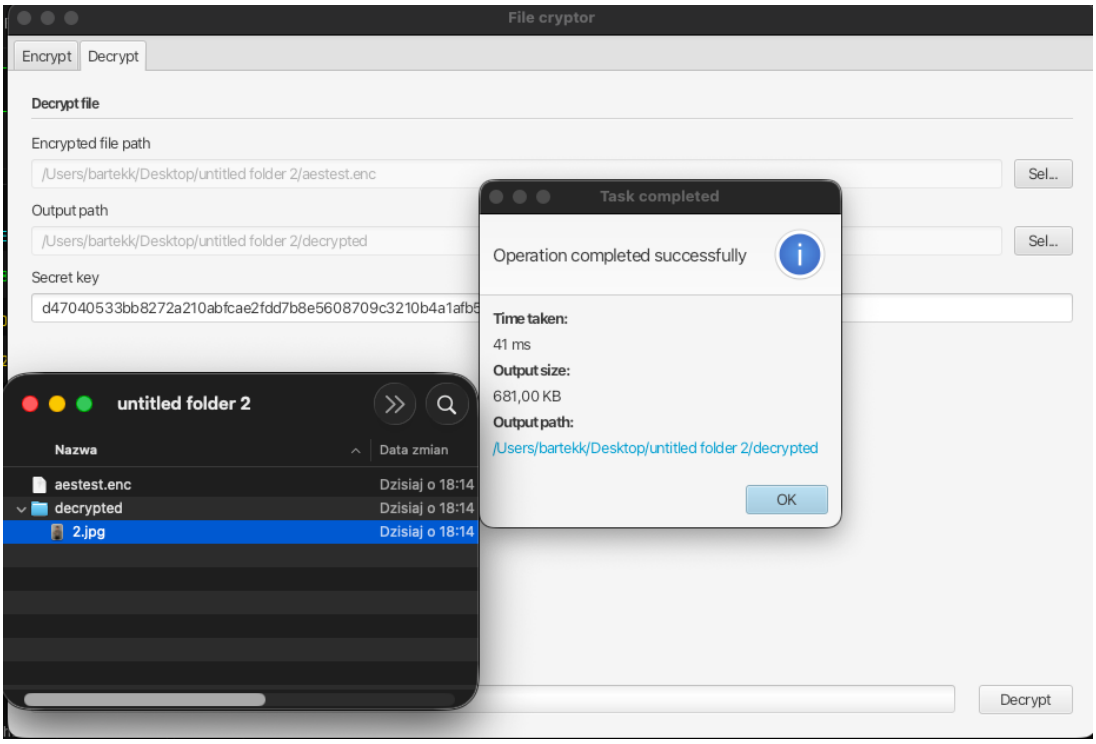
- Czas wykonania operacji (w milisekundach lub sekundach)
- Rozmiar wygenerowanego pliku
- Ścieżka do pliku wyjściowego (klikalna, otwiera folder w systemie plików)



Rysunek 7: Komunikat sukcesu po szyfrowaniu - algorytm Caesar Cipher



Rysunek 8: Komunikat sukcesu po deszyfrowaniu



Rysunek 9: Pełny widok aplikacji z komunikatem sukcesu deszyfrowania

6 Wyniki testów i wydajność

6.1 Testy funkcjonalne

Przeprowadzono szereg testów funkcjonalnych aplikacji, sprawdzając działanie wszystkich trzech algorytmów kryptograficznych na różnych typach plików.

6.2 Wyniki testów wydajności

Na podstawie rzeczywistych testów przeprowadzonych w aplikacji, uzyskano następujące wyniki wydajności:

Tabela 3: Wyniki testów wydajności - szyfrowanie

Algorytm	Rozmiar pliku	Czas szyfrowania	Przepływność
Caesar Cipher	528.80 KB	2.72 s	0.19 MB/s
XOR Cipher	528.81 KB	62 ms	8.53 MB/s
AES	528.81 KB	62 ms	8.53 MB/s

Tabela 4: Wyniki testów wydajności - deszyfrowanie

Algorytm	Rozmiar pliku	Czas deszyfrowania	Przepływność
AES	681.00 KB	41 ms	16.61 MB/s
AES (mały plik)	68.96 KB	361 ms	0.19 MB/s

6.3 Analiza wyników

Z przeprowadzonych testów wynika, że:

- **AES** jest najszybszym algorytmem - osiąga najwyższą przepływność zarówno przy szyfrowaniu (8.53 MB/s) jak i deszyfrowaniu (16.61 MB/s dla większych plików)
- **Caesar Cipher** jest najwolniejszym algorytmem - dla pliku o rozmiarze 528.80 KB czas szyfrowania wyniósł 2.72 sekundy, co daje przepływność zaledwie 0.19 MB/s
- **XOR Cipher** osiąga podobną wydajność do AES dla testowanego pliku
- Wydajność deszyfrowania AES jest wyższa niż szyfrowania, co jest typowe dla algorytmów symetrycznych

Warto zauważyć, że wyniki dla małych plików mogą być obciążone narzutem związanym z inicjalizacją algorytmu i operacjami wejścia/wyjścia, dlatego dla małych plików czas może być nieproporcjonalnie długi w stosunku do rozmiaru.

6.4 Testy jednostkowe

Aplikacja zawiera testy jednostkowe dla wszystkich implementacji algorytmów kryptograficznych, które weryfikują poprawność działania szyfrowania i deszyfrowania.

```
@Test
void encrypt() throws IOException {
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    cezarCryptor.encrypt(new ByteArrayInputStream(original), out);
    assertArrayEquals(expectedEncrypted, out.toByteArray());
}

@Test
void decrypt() throws IOException {
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    cezarCryptor.decrypt(new ByteArrayInputStream(expectedEncrypted),
   out);
    assertArrayEquals(original, out.toByteArray());
}
```

Listing 5: Przykład testu jednostkowego dla Caesar Cipher

Wszystkie testy jednostkowe przechodzą pomyślnie, co potwierdza poprawność implementacji algorytmów.

7 Bezpieczeństwo

7.1 Zagadnienia bezpieczeństwa

7.1.1 Bezpieczeństwo kluczy

- Klucze nie są przechowywane w pamięci aplikacji po zakończeniu operacji
- Parametry kluczy są walidowane przed użyciem
- AES wykorzystuje bezpieczny generator liczb losowych (SecureRandom) dla wektora inicjalizującego (IV)

- Każde szyfrowanie AES używa unikalnego IV, co zapewnia, że te same dane zaszyfrowane tym samym kluczem dają różne wyniki

7.1.2 Zabezpieczenia implementacyjne

- Użycie bezpiecznych trybów operacyjnych (CBC dla AES z PKCS5Padding)
- Prawidłowa obsługa błędów kryptograficznych
- Unikanie wycieków informacji przez strony kanały
- Walidacja nagłówków plików zaszyfrowanych przed deszyfrowaniem

7.1.3 Ograniczenia bezpieczeństwa

Należy pamiętać, że:

- Caesar Cipher i XOR Cipher oferują niski poziom bezpieczeństwa i nie powinny być używane do ochrony wrażliwych danych
- AES jest jedynym algorytmem oferującym przemysłowy standard bezpieczeństwa
- Aplikacja nie implementuje funkcji haszowania do weryfikacji integralności plików

8 Wnioski i rekomendacje

8.1 Cechy aplikacji

File Crypto jest funkcjonalną aplikacją do szyfrowania plików, która charakteryzuje się następującymi cechami:

- Intuicyjny interfejs użytkownika oparty na JavaFX
- Wsparcie dla trzech różnych algorytmów kryptograficznych (Caesar Cipher, XOR Cipher, AES)
- Wysoka wydajność algorytmu AES
- Możliwość szyfrowania wielu plików jednocześnie
- Prawidłowa obsługa błędów z czytelnymi komunikatami
- Szczegółowe statystyki po zakończeniu operacji (czas wykonania, rozmiar pliku)
- Automatyczne tworzenie archiwów ZIP przed szyfrowaniem
- Konfigurowalne parametry dla każdego algorytmu
- Wyświetlanie postępu operacji w czasie rzeczywistym
- Walidacja danych wejściowych przed rozpoczęciem operacji
- Obsługa różnych formatów kluczy (tekst, heksadecymalny)
- Możliwość otwarcia folderu wyjściowego bezpośrednio z aplikacji

8.2 Możliwości rozwoju

1. **Optymalizacja wydajności:** Zoptymalizować implementację Caesar Cipher, która obecnie jest nieefektywna
2. **Dodatkowe algorytmy:** Rozważyć dodanie wsparcia dla algorytmów asymetrycznych (RSA, ECC) do wymiany kluczy
3. **Weryfikacja integralności:** Implementacja funkcji haszowania (SHA-256, SHA-3) do weryfikacji integralności plików
4. **Obsługa dużych plików:** Dodanie możliwości przetwarzania strumieniowego dla bardzo dużych plików
5. **Ulepszenie zarządzania kluczami:** System zarządzania kluczami z możliwością ich przechowywania w bezpiecznym magazynie
6. **Kompresja:** Opcjonalna kompresja przed szyfrowaniem dla lepszego wykorzystania przestrzeni dyskowej
7. **Historia operacji:** Logowanie historii operacji szyfrowania/deszyfrowania
8. **Multi-threading:** Wykorzystanie wielu wątków do równoległego przetwarzania wielu plików

9 Podsumowanie

Aplikacja File Crypto została pomyślnie zaimplementowana i przetestowana. Oferuje funkcjonalny interfejs użytkownika oraz wsparcie dla trzech algorytmów kryptograficznych. Testy wykazały, że algorytm AES charakteryzuje się najlepszą wydajnością, podczas gdy Caesar Cipher jest najwolniejszy. Aplikacja poprawnie obsługuje błędy i dostarcza szczegółowe informacje o wykonanych operacjach.

Aplikacja spełnia podstawowe wymagania funkcjonalne i może być wykorzystana do szyfrowania plików w codziennym użytku, przy czym należy pamiętać o ograniczeniach bezpieczeństwa prostszych algorytmów (Caesar Cipher, XOR Cipher) i preferować użycie AES dla wrażliwych danych.

10 Instrukcje

10.1 Instrukcja użytkownika

10.1.1 Szyfrowanie plików

1. Otwórz aplikację File Crypto
2. Przejdź do zakładki "Encrypt"
3. Dodaj pliki do zaszyfrowania (przeciągnij i upuść lub użyj przycisku "Add files...")
4. Wybierz algorytm szyfrowania z listy rozwijanej
5. Skonfiguruj parametry algorytmu (klucz, przesunięcie)

6. Wybierz ścieżkę wyjściową i wprowadź nazwę pliku
7. Kliknij przycisk "Encrypt"
8. Po zakończeniu operacji sprawdź szczegóły w oknie dialogowym

10.1.2 Deszyfrowanie plików

1. Otwórz aplikację File Crypto
2. Przejdź do zakładki "Decrypt"
3. Wybierz zaszyfrowany plik (rozszerzenie .enc)
4. Wybierz ścieżkę wyjściową dla zdeszyfrowanych plików
5. Wprowadź klucz szyfrowania (w formacie odpowiednim dla użytego algorytmu)
6. Kliknij przycisk "Decrypt"
7. Po zakończeniu operacji sprawdź szczegóły w oknie dialogowym